Write a Python Program to display all positions of substring in a given main string.

In [8]:
```
s=input("Enter main string:")
subs=input("Enter sub string:")
flag=False
pos=-1
n=len(s)
c = 0
while True:
    pos=s.find(subs,pos+1,n)
    if pos==-1:
        break
    c = c+1
    print("Found at position",pos)
    flag=True
if flag==False:
    print("Not Found")
print('The number of occurrences : ',c)
```

```
Enter main string:abcdabcdacbdaas
Enter sub string:a
Found at position 0
Found at position 4
Found at position 8
Found at position 12
Found at position 13
The number of occurrences :  5
```

In [9]:
```
s=input("Enter main string:")
subs=input("Enter sub string:")
flag=False
pos=-1
n=len(s)
c = 0
while True:
    pos=s.find(subs,pos+1,n)
    if pos==-1:
        break
    c = c+1
    print("Found at position",pos)
    flag=True
if flag==False:
    print("Not Found")
print('The number of occurrences : ',c)
```

```
Enter main string:abcdabcdabcd
Enter sub string:z
Not Found
The number of occurrences :  0
```

Program to merge characters of 2 strings into a single string by taking characters alternatively.

In [10]:
```
s1=input("Enter First String:")
s2=input("Enter Second String:")
output=''
i,j=0,0
while i<len(s1) or j<len(s2):
```

```
    output = output + s1[i]
    i = i + 1
    output = output + s2[j]
    j = j + 1
print(output)
```

```
Enter First String:abc
Enter Second String:123
a1b2c3
```

Write a program to sort the characters of the string and first alphabet symbols followed by numeric values

In [11]:
```
s=input("Enter Some String:")
s1=s2=output=''
for x in s:
    if x.isalpha():
        s1=s1+x # All alphabets in s1
    else:
        s2=s2+x # All digits in s2

for x in sorted(s1): # In s1, whatever the content is there which will be sorted.
    output=output+x
for x in sorted(s2): # # In s2, whatever the content is there which will be sorted
    output=output+x
print(output)
```

```
Enter Some String:B4A1D3
ABD134
```

Write a program for the following requirement:

input: a4b3c2

output: aaaabbbcc

In [12]:
```
s=input("Enter Some String:")
output=''
for x in s:
    if x.isalpha():
        output=output+x
        previous=x
    else:
        output = output + previous*(int(x)-1)
print(output)
```

```
Enter Some String:a4b3c2
aaaabbbcc
```

Program to perform a circular shift on a list to the right

In [14]:
```
l=[1,2,3,4,5,6,7]
s=int(input("Enter Shift Amount:"))
x=l[:]
x=l[-s:]+l[:-s]
print("Original List:",l)
print("Shifted List:",x)
```

```
Enter Shift Amount:2
Original List: [1, 2, 3, 4, 5, 6, 7]
```

```
Shifted List: [6, 7, 1, 2, 3, 4, 5]
```

In [16]:
```python
def circular_shift_right(my_list, shift_amount):
    shift_amount = shift_amount % len(my_list)
    my_list[:] = my_list[-shift_amount:] + my_list[:-shift_amount]

numbers = [1, 2, 3, 4, 5, 6, 7]
shift_amount = 3

print("Original List:", numbers)
circular_shift_right(numbers, shift_amount)
print(f"List after circular shift by {shift_amount} to the right:", numbers)
```

```
Original List: [1, 2, 3, 4, 5, 6, 7]
List after circular shift by 3 to the right: [5, 6, 7, 1, 2, 3, 4]
```

Program to transpose a matrix represented as a list of lists

In [18]:
```python
matrix = [[1, 2, 3],[4, 5, 6],[7, 8, 9]]
print("Original Matrix")
for i in matrix:
    print(i)
print()
print("Transpose Matrix")
for i in range(len(matrix[0])):
        new=[row[i] for row in matrix]
        print(new)
```

```
Original Matrix
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

Transpose Matrix
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
```

Write a Program in Python to Print Elements With Frequency Greater Than a Given Value k

In [22]:
```python
List1 = [1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 5, 5, 5, 6, 7]
k = 2
print("Original List:", str(List1))

#creating empty list
res_list = []

#iterating through list count and add the item to new list if count > k
for i in List1:
   counter = List1.count(i)
   if counter > k and i not in res_list:
     res_list.append(i)

print("The list items with count >",k," = ", res_list)
```

```
Original List: [1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 5, 5, 5, 6, 7]
The list items with count > 2  =  [1, 2, 5]
```

Write a Python Program to Remove All Occurrences of an Element From a Given List

In [23]:
```
#Remove All Occurrences of an Element
List1 = [22, 24, 30, 22, 45, 67, 22, 30, 45]
item_to_remove = 22

print("The original list is : " + str(List1))

#function to remove all occurances
def remove_an_item(test_list, value):

  # remove the item for all its occurrences
  for x in test_list:
    if(x == value):
      test_list.remove(x)
  return test_list

# calling the function remove_items()
res_list = remove_an_item(List1, item_to_remove)

# updated list after removing 22
print("Updated list after remove operation: " + str(res_list))
```

```
The original list is : [22, 24, 30, 22, 45, 67, 22, 30, 45]
Updated list after remove operation: [24, 30, 45, 67, 30, 45]
```

Get the index of the max value in the list using the max() and index()

In [24]:
```
# Defining a list
l = [1, 4, 3, 7, 8, 10, 2]

# Calculating the maximum element in the list
m = max(l)

# Finding the index of the maximum element
print("Index of the max element in a list is", l.index(m))
```

```
Index of the max element in a list is 5
```

In [27]:
```
# Defining a list
l = [1, 4, 3, 7, 8, 10, 2]

# Calculating the maximum element in the list
m = max(l)

# Finding the index of the maximum element
i = 0
while(i < len(l)):
    if l[i] == m:
        print("Index of the max element in a list is", i)
        break
    i += 1
```

```
Index of the max element in a list is 5
```

Python code to find the index at which the element of two list doesn't match

In [28]:
```
# List initialisation
Input1 = [1, 2, 3, 4]
Input2 = [1, 5, 3, 6]

# Index initialisation
y = 0

# Output list initialisation
Output = []

# Using iteration to find
for x in Input1:
    if x != Input2[y]:
        Output.append(y)
    y = y + 1

# Printing output
print(Output)
```

```
[1, 3]
```

Write a python program to check the validity of a Password

Primary conditions for password validation:

1. Minimum 8 characters.
2. The alphabet must be between [a-z]
3. At least one alphabet should be of Upper Case [A-Z]
4. At least 1 number or digit between [0-9]
5. At least 1 character from [_ or @ or $]

Examples: Input: Ram@_f1234 Output: Valid Password

```
Input: Rama_fo$ab
Output: Invalid Password
Explanation: Number is missing

Input: Rama#fo9c
Output: Invalid Password
Explanation: Must consist from _ or @ or \$
```

In [1]:
```
def password_check(password):
    l, u, p, d = 0, 0, 0, 0
    if len(password) >= 8:
        for i in password:
        # counting lowercase alphabets
            if (i.islower()):
                l+=1
        # counting uppercase alphabets
            if (i.isupper()):
                u+=1
        # counting digits
```

```
            if (i.isdigit()):
                d+=1
        # counting the mentioned special characters
            if(i=='@'or i=='$' or i=='_'):
                p+=1
        if (l>=1 and u>=1 and p>=1 and d>=1 and l+p+u+d==len(password)):
            print("valid")
        else:
            print("invalid")
    else:
        print("invalid")
password_check("Ram@_f1234")
```

```
valid
```

Write a program to check if two strings are balanced. For example, strings s1 and s2 are balanced if all the characters in the s1 are present in s2 and length of s1 & s2 should be same. The character's position doesn't matter.

Example :

s1 = hello s2 = olleh Balanced

In [2]:
```
s1 = "hello"
s2 = "olleh"
    # Check if lengths are equal
if len(s1) != len(s2):
        print("not balanced")
# Check if all characters in s1 are present in s2
for char in s1:
    if char not in s2:
        print("not balanced")
print("balanced")
```

```
balanced
```

Write a python code to explain map in Python program

In [3]:
```
# Example 1: Doubling each element in a list using map
numbers = [1, 2, 3, 4, 5]
doubled_numbers = list(map(lambda x: x * 2, numbers))
print("Doubled Numbers:", doubled_numbers)
# Example 2: Converting strings to uppercase using map
words = ["apple", "banana", "cherry"]
uppercase_words = list(map(str.upper, words))
print("Uppercase Words:", uppercase_words)
```

```
Doubled Numbers: [2, 4, 6, 8, 10]
Uppercase Words: ['APPLE', 'BANANA', 'CHERRY']
```

Write a python program with user defined function that reads the words from paragraph and stores them as keys in a dictionary and count the frequency of it as a value

Input string: "Dog the quick brown fox jumps over the lazy dog"

Output: {'the': 2, 'jumps': 1, 'brown': 1, 'lazy': 1, 'fox': 1, 'over': 1, 'quick': 1, 'dog': 2} `

In [4]:
```
def word_count(str):
    counts = dict()
    words = str.split()
    for word in words:
        if word in counts:
            counts[word] += 1
        else:
            counts[word] = 1
    return counts
print( word_count('the quick brown fox jumps over the lazy dog.'))
```

```
{'the': 2, 'quick': 1, 'brown': 1, 'fox': 1, 'jumps': 1, 'over': 1, 'lazy': 1, 'dog.':
1}
```

Write a Python Program using function to count number of strings where the string length is 3 or more and the first and last character are same from a given list of string.

Example :

Input: ['abc','xyz','aba','2112','123451','12345'] Output: 3

In [5]:
```
strings=['abc', 'xyz', 'aba', '2112', '123451', '12345']
count = 0
for string in strings:
        if len(string) >= 3 and string[0] == string[-1]:
            count += 1
print("count=",count)
```

```
count= 3
```

Given a list L of size N. You need to count the number of special elements in the given list. An element is special if removal of that element makes the list balanced. The list will be balanced if sum of even index elements is equal to the sum of odd elements. Also print the updated lists after removal of special elements.

Example:

Example 1: Input: L=[5, 5, 2, 5, 8] Output: Original List: [5, 5, 2, 5, 8] Index to be removed is: 0 List after removing index 0 : [5, 2, 5, 8] Original List: [5, 5, 2, 5, 8] Index to be removed is: 1 List after removing index 1 : [5, 2, 5, 8] Total number of special elements: 2 Explaination: If we delete L[0] or L[1], list will be balanced. [5, 2, 5, 8] (5+5) = (2+8) So L[0] and L[1] are special elements, So Count is 2. After removal of the special elements, list will be: [5, 2, 5, 8] Example 2: Input: L=[2,1,6,4] Output: Original List: [2, 1, 6, 4] Index to be removed is: 1 List after removing index 1 : [2, 6, 4] Total Number of Special elements: 1 Explaination: If we delete L[1] from list : [2,6,4] (2+4) = (6) Here only 1 special element. So Count is 1. After removal of special element, list will be : [2,6,4]

In [31]:
```
l=eval(input("enter list"))
print("Original List:",l)
print()
count=0
for i in range(0,len(l)):
    c=l.copy()
    c.pop(i)
    sum1=sum(c[0::2])
    sum2=sum(c[1::2])
    if sum1==sum2:
```

```
        print("index to be removed is: ",i)
        count+=1
        print("List after removing index ",i,"is ",l[0:i]+l[i+1:])
        print()
print("Total Number of Special elements:",count)
```

```
enter list[5,5,2,5,8]
Original List: [5, 5, 2, 5, 8]

index to be removed is:  0
List after removing index  0 is  [5, 2, 5, 8]

index to be removed is:  1
List after removing index  1 is  [5, 2, 5, 8]

Total Number of Special elements: 2
```

Write Python Program to create a dictionary with the key as the first character and value as a list of words starting with that character.

Example:

Input: Don't wait for your feelings to change to take the action. Take the action and your feelings will change

Output: {'D': ['Don't'], 'w': ['wait', 'will'], 'f': ['for', 'feelings', 'feelings'], 'y': ['your', 'your'], 't': ['to', 'to', 'take', 'the', 'the'], 'c': ['change', 'change'], 'a': ['action.', 'action', 'and'], 'T': ['Take']}

In [6]:
```
sentence = "Don't wait for your feelings to change to take the action."
words = sentence.split()
result_dict = {}
for word in words:   # Use the first character of the word as the key
        key = word[0]     # Check if the key is already present in the dictionary
        if key in result_dict:
            result_dict[key].append(word)
        else:
            result_dict[key] = [word]
print(result_dict)
```

```
{'D': ['Don't'], 'w': ['wait'], 'f': ['for', 'feelings'], 'y': ['your'], 't': ['to', 't
o', 'take', 'the'], 'c': ['change'], 'a': ['action.']}
```

In [7]:
```
l=[1,2,3,[1,2],"abcd"]
print(l[-1:-3:-1][-1][-2])
```

```
1
```

Replace words from Dictionary. Given String, replace it's words from lookup dictionary.

Example 1:

Input:

test_str = 'CampusX best for DS students.'

repl_dict = {"best" : "is the best channel", "DS" : "Data-Science"}

Output: CampusX is the best channel for Data-Science students.

Example 2:

Input:

test_str = 'CampusX best for DS students.'

repl_dict = {"good" : "is the best channel", "ds" : "Data-Science"}

Output: CampusX best for DS students.

In [8]:
```
test_str = 'CampusX best for DS students.'
repl_dict = {"best" : "is the best channel", "DS" : "Data-Science"}

res = []
for i in test_str.split():
    if i in repl_dict:
        res.append(repl_dict[i])
    else:
        res.append(i)
print(" ".join(res))
```

```
CampusX is the best channel for Data-Science students.
```

Creating a simple calculator using lambda functions

In [ ]:
```
calculator = {
    'add': lambda x, y: x + y,
    'subtract': lambda x, y: x - y,
    'multiply': lambda x, y: x * y,
    'divide': lambda x, y: x / y if y != 0 else "Cannot divide by zero"
}

result_add = calculator['add'](10, 5)
result_multiply = calculator['multiply'](4, 6)

print(result_add)        # Output: 15
print(result_multiply)   # Output: 24
```

Sorting a list of tuples by the second element using lambda

Syntax sorted(iterable, key=key, reverse=reverse)

Parameter Values

Parameter Description

iterable- Required. The sequence to sort, list, dictionary, tuple etc.

key- Optional. A Function to execute to decide the order. Default is None

reverse- Optional. A Boolean. False will sort ascending, True will sort descending. Default is False

In [5]:
```
pairs = [(1, 5), (2, 3), (3, 8), (4, 1)]
sorted_pairs = sorted(pairs,key=lambda x: x[1])

print(sorted_pairs)
```

```
[(4, 1), (2, 3), (1, 5), (3, 8)]
```

Sorting a list of strings by the length of each string

In [6]:
```
words = ['apple', 'banana', 'kiwi', 'orange', 'grape']
sorted_words = sorted(words, key=lambda x: len(x))

print(sorted_words)
```

```
['kiwi', 'apple', 'grape', 'banana', 'orange']
```

Using lambda with map to capitalize the first letter of each word in a list

In [ ]:
```
words = ['python', 'is', 'awesome']
capitalized_words = list(map(lambda x: x.capitalize(), words))

print(capitalized_words)
# Output: ['Python', 'Is', 'Awesome']
```

Define functions to calculate the cost of customized pizzas and place orders.

The calculate_pizza_price function computes the cost based on the pizza's size, toppings, and cheese selected. It uses predefined prices for sizes(small(50),medium(100),large(200)), toppings (categorized as 20(corn,tomato,onion,capsicum) or 50(mushroom,olives,brocolli), and adds an additional cost(50) for each cheese type(feta,mozarella,cheddar).

The order_pizza function prompts users to customize their pizzas by selecting size, toppings (with associated prices), and cheese. It creates a list of tuples representing each ordered pizza's details.

The print_bill function displays the details of each pizza ordered, including size, toppings, cheese, and individual pizza costs, summing up to the total bill amount.

Finally, the main program executes the pizza ordering process, collects the order, and prints out the bill for the customer.

In [4]:
```
def calculate_pizza_price(size, toppings, cheese):
    cost = 0

    # Calculate pizza base price based on size
    if size == 'small':
        cost += 50
    elif size == 'medium':
        cost += 100
    else:
        cost += 200

    # Calculate topping prices
```

```
    topping_prices_20 = ['corn', 'tomato', 'onion', 'capsicum']
    topping_prices_50 = ['mushroom', 'olives', 'broccoli']

    for topping in toppings:
        cost += 20 if topping in topping_prices_20 else 50

    # Calculate cheese price
    cost += 50 * len(cheese)

    return cost

def order_pizza():
    pizzas = []
    number = int(input("How many pizzas you want to order: "))

    for i in range(number):
        toppings = []
        cheese = []

        print('Customize Pizza', i + 1)
        print('Select Pizza Size: small, medium or large')
        size = input('Select size: ')
        print('''Select Toppings:
        20 for 'corn', 'tomato', 'onion', 'capsicum'
        50 for 'mushroom', 'olives', 'broccoli' ''')
        toppings_count = int(input('How many toppings: '))
        for _ in range(toppings_count):
            toppings.append(input('Enter toppings: '))
        print('Select Cheese: mozarrella, feta, cheddar')
        cheese_count = int(input('How many cheese: '))
        for _ in range(cheese_count):
            cheese.append(input('Enter cheese: '))
        print()
        pizza_cost = calculate_pizza_price(size, toppings, cheese)
        pizzas.append((size, toppings, cheese, pizza_cost))

    return pizzas

def print_bill(pizzas):
    total = 0
    count = 1

    for pizza in pizzas:
        print('Pizza', count)
        print("Size:", pizza[0])
        print("Toppings:", pizza[1])
        print("Cheese:", pizza[2])
        print("Pizza cost:", pizza[3])
        total += pizza[3]
        count += 1

    print('Total bill amount:', total)

# Main program
pizza_order = order_pizza()
print_bill(pizza_order)
```

```
How many pizzas you want to order: 2
Customize Pizza 1
Select Pizza Size: small, medium or large
```

```
Select size: small
Select Toppings:
        20 for 'corn', 'tomato', 'onion', 'capsicum'
        50 for 'mushroom', 'olives', 'broccoli'
How many toppings: 3
Enter toppings: corn
Enter toppings: tomato
Enter toppings: onion
Select Cheese: mozarrella, feta, cheddar
How many cheese: 1
Enter cheese: mozarella

Customize Pizza 2
Select Pizza Size: small, medium or large
Select size: small
Select Toppings:
        20 for 'corn', 'tomato', 'onion', 'capsicum'
        50 for 'mushroom', 'olives', 'broccoli'
How many toppings: 1
Enter toppings: corn
Select Cheese: mozarrella, feta, cheddar
How many cheese: 1
Enter cheese: mozarella

Pizza 1
Size: small
Toppings: ['corn', 'tomato', 'onion']
Cheese: ['mozarella']
Pizza cost: 160
Pizza 2
Size: small
Toppings: ['corn']
Cheese: ['mozarella']
Pizza cost: 120
Total bill amount: 280
```

In [ ]: